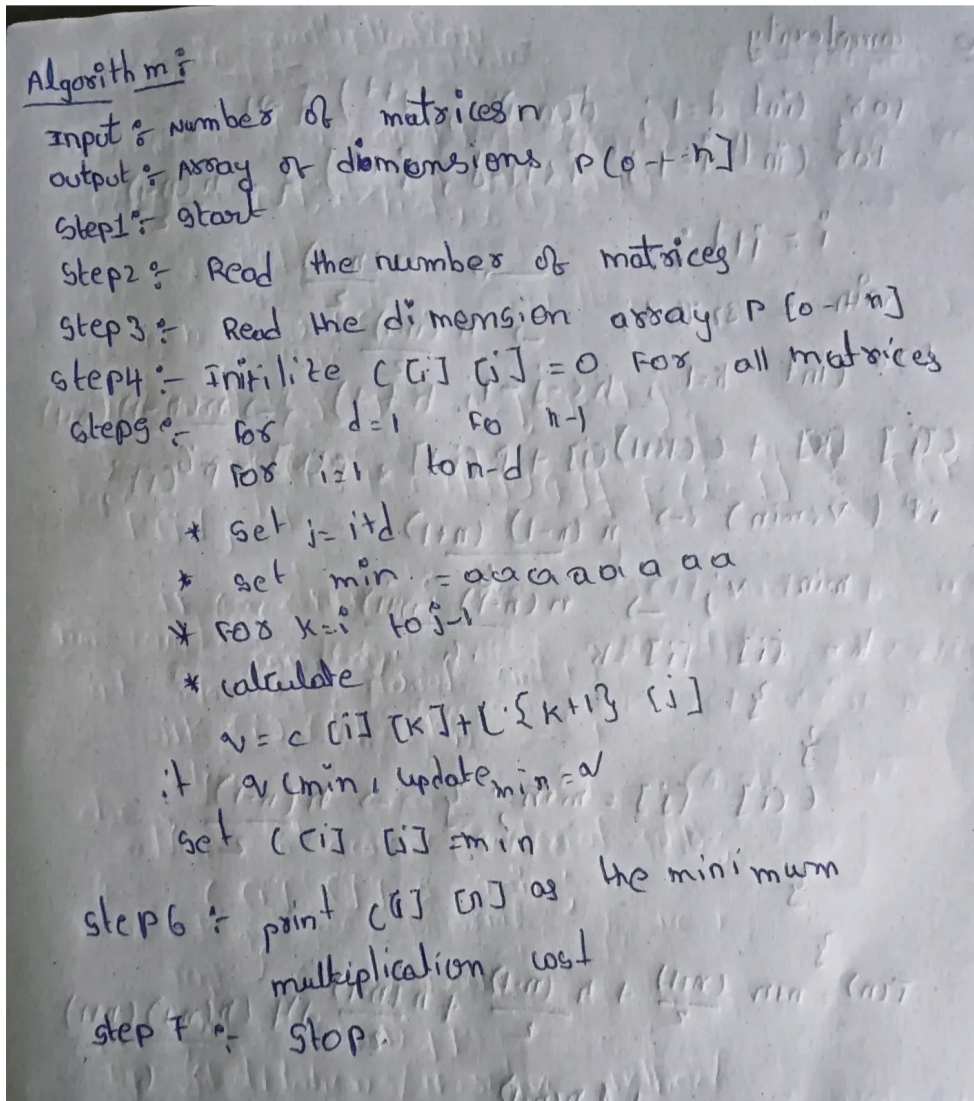


PRACTICAL-6

Aim : Implementation of chain matrix multiplication using dynamic programming.

Algorithm :



Algorithm:
Input: number of matrices n
Output: Array of dimensions $p[0..n]$
Step 1: Start
Step 2: Read the number of matrices
Step 3: Read the dimension array $p[0..n]$
Step 4: Initialize $C[i][j] = 0$ for all matrices
Step 5: for $d = 1$ to $n-1$
 for $i = 1$ to $n-d$
 * set $j = i+d$
 * set $min = \infty$
 * for $k = i$ to $j-1$
 * calculate
 $v = C[i][k] + L[k+1][j]$
 if $v < min$, update $min = v$
 set $C[i][j] = min$
Step 6: print $C[1][n]$ as the minimum multiplication cost
Step 7: Stop

Program / Code :

```
#include <climits>
#include <iostream>
```

```
using namespace std;
```

NAME: Raja Kiran
ENROLLMENT NO: 92400118654

```
void printTable(int m[][100], int n) {
    cout << "\nMinimum Cost Table:\n";
    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            if (m[i][j] == INT_MAX)
                cout << "-\t";
            else
                cout << m[i][j] << "\t";
        }
        cout << "\n";
    }
}
```

```
int matrixChainOrder(int p[], int n) {
    int m[100][100];

    for (int i = 1; i < n; i++) {
        m[i][i] = 0;
    }

    for (int L = 2; L < n; L++) {
        for (int i = 1; i < n - L + 1; i++) {
            int j = i + L - 1;
            m[i][j] = INT_MAX;

            for (int k = i; k < j; k++) {
                int q = m[i][k] + m[k + 1][j]
                    + p[i - 1] * p[k] * p[j];

                if (q < m[i][j]) {
                    m[i][j] = q;
                }
            }
        }
    }

    printTable(m, n);
    return m[1][n - 1];
}
```

```
int main() {
    int arr[] = {3, 5, 10, 7, 2};
    int n = sizeof(arr) / sizeof(arr[0]);
}
```

```
cout << "Minimum number of multiplications is "  
    << matrixChainOrder(arr, n) << endl;  
  
    return 0;  
}
```

Output :

```
Minimum number of multiplications is  
Minimum Cost Table:  
0   150 360 270  
0   0   350 240  
0   0   0   140  
0   0   0   0  
270
```

Time Complexity :

Time complexity

```

for (int d=1; d<n; d++) {
    for (int i=1; i<n-d; i++) {
        i = i+d;
        min = 32767;
        for (int k=1; k<=j-i; k++) {
            v = (C[i][k] + (C[k+1][j] + P[i-1] * P[k] * P[j]));
            if (v < min) min = v;
            k = [i] [j] = k;
        }
        C[i][j] = min;
    }
}
T(n) = n * n * (n+1)/2 + n * (n-1)/2 + n * (n-1)/2 + 4 * (n-1)(n+1)/6
T(n) = O(n^3)
    
```

1. BEST CASE:

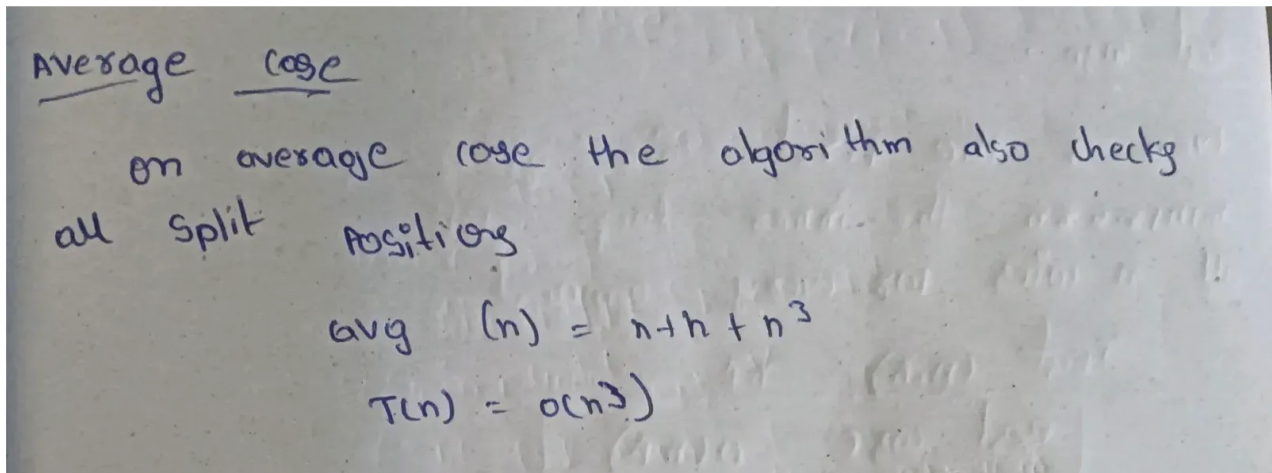
Best case :-
Even in the best case, the algorithm checks all possible split positions

$$T(n) = n + n^2 + n^3$$

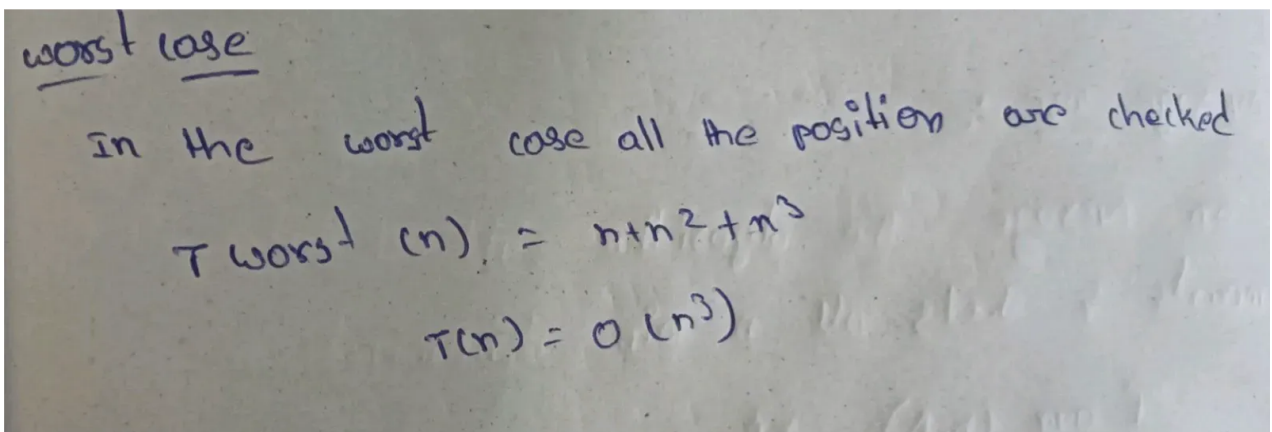
$$= O(n^3)$$

$$T(n) = O(n^3)$$

2.AVERAGE CASE:



3.WORST CASE:



Conclusion :

The Matrix Chain Multiplication problem was successfully implemented using the Dynamic Programming approach. The algorithm determines the optimal order of multiplying a sequence of matrices by minimizing the total number of scalar multiplications required. Thus, dynamic programming helps reduce the computational cost by avoiding unnecessary repeated calculations.